

LAB PRACTICUM: OpenMP Multi-Core Scaling in Python

Hands-Off Benchmark Practicum • 60 Minutes • Autonomous Execution

Student Name: Nuriya Sultanseitova | Student ID: 230103360

Hardware / CPU: Apple M4, 10 cores | Date & Section: 02.10.2026 & 08-P

Challenge 1: The Core Speedometer & Amdahl's Law

Benchmark Results

Threads	Time (seconds)	Speedup	Efficiency
1	0.6689	1.00x	100.0%
2	0.3408	1.96x	98.1%
4	0.2057	3.25x	81.3%
8	0.1558	4.29x	53.7%
10	0.1500	4.46x	44.6%

A. Baseline and maximum thread result

My baseline time with 1 thread was **0.6689 seconds**. With the maximum number of threads (10), the runtime was **0.1500 seconds**. The speedup was **4.46x** and the efficiency was **44.6%**.

B. Why is the warm-up call required?

The warm-up call is needed because Numba compiles the function when it runs for the first time. If we include this compilation time in the benchmark, the result will not be accurate. So, the warm-up runs the compilation before we start measuring the real execution time.

C. Did the parallel efficiency stay at 100%?

No, the efficiency did not stay at 100%. With 2 threads it was 98.1%, but with 10 threads it decreased to 44.6%. More threads have to share CPU and memory resources, and there is also some overhead when threads work together. Because of this, adding more threads does not give the same

performance improvement every time.

D. Why does `inside_circle += 1` not corrupt the count?

Numba understands `inside_circle += 1` as a reduction operation. Each thread can calculate its own part of the count. At the end, these results are combined together. It works like an OpenMP `reduction(+:inside_circle)` operation.

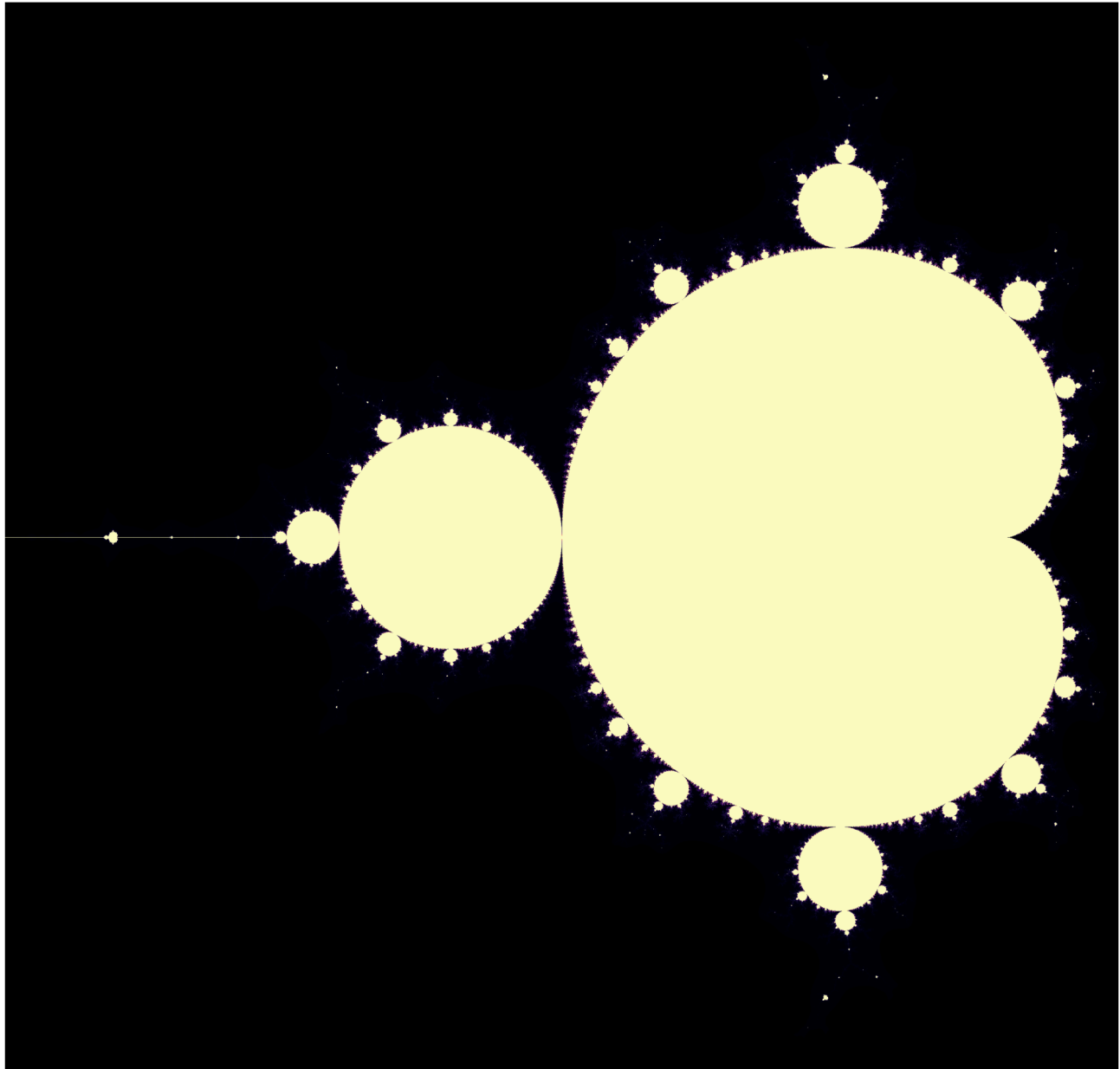
These are the four required technical questions for Challenge 1.

Challenge 2: Load Imbalance & Dynamic Scheduling

Benchmark Results

Method	Time
Row-Parallel	0.889 s
Column-Parallel	0.943 s

Mandelbrot 2500x2500 (Render: 0.89s)



A. Row vs Column

On my machine, the **row-parallel version was faster**. It took 0.889 seconds, while the column-parallel version took 0.943 seconds. The difference is not very big, but the row version worked better in my test.

B. C-contiguous memory and CPU cache

NumPy arrays use C-contiguous, or row-major, memory layout. It means that elements from the same row are stored close to each other in memory. Because of this, processing rows can use CPU cache

more efficiently. When we access data by columns, the CPU may need to jump between different memory locations more often.

C. Load imbalance and dynamic scheduling

Different parts of the Mandelbrot set need different amounts of work. Some points finish after only a few iterations, while other points need many iterations. Because of this, one thread can finish early while another thread is still working.

Dynamic scheduling can help with this problem. Instead of giving one large fixed part to every thread, new work can be given to threads when they become free. This can make the workload more balanced.

D. Mandelbrot Output

The program successfully created the **mandelbrot_output.png** file. I attached the generated image with my completed work.

[INSERT **mandelbrot_output.png** HERE]

The task specifically asks for the exported image to be attached with the completed task sheet.
OpenMP_Python_Lab_Practicum

Challenge 3: Stencil Computation & Memory Bandwidth

Benchmark Results

Precision	Runtime	Throughput
float64	0.094 s	7169.37 Megacells/sec
float32	0.063 s	10699.40 Megacells/sec

A. Benchmark Result

With **float64**, the heat diffusion benchmark finished in **0.094 seconds**. The throughput was **7169.37 Megacells/sec**.

B. float64 vs float32

After changing the array type from **float64** to **float32**, the runtime decreased from 0.094 seconds to 0.063 seconds.

The improvement was approximately:

$$0.094 / 0.063 = 1.49x$$

So, `float32` was about **1.49 times faster** in my test.

One reason is that `float32` uses less memory than `float64`. This means that less data has to move between memory and the CPU. Since this task depends a lot on memory access, reducing the amount of data can improve the performance.

C. Memory Roofline Model

Adding twice as many CPU cores does not always give a 2x speedup. The heat stencil needs to read and write a lot of data from memory. When the memory bandwidth is already fully used, adding more CPU cores cannot increase the memory speed. The cores have to share the same memory bandwidth, so the performance improvement becomes smaller.

These are the required A–C parts for Challenge 3.

Student Result Scorecard

Benchmark Challenge	Active Threads	Array Size / Samples	Time (seconds)	Speedup	Efficiency
Challenge 1: Monte Carlo	1 Thread (Base)	120,000,000	0.6689	1.00x	100.0%
Challenge 1: Monte Carlo	2 Threads	120,000,000	0.3408	1.96x	98.1%
Challenge 1: Monte Carlo	4 Threads	120,000,000	0.2057	3.25x	81.3%
Challenge 1: Monte Carlo	Max Threads (10)	120,000,000	0.1500	4.46x	44.6%
Challenge 2: Mandelbrot (Rows)	Max Threads	2500 × 2500	0.889	N/A	N/A
Challenge 2: Mandelbrot (Cols)	Max Threads	2500 × 2500	0.943	N/A	N/A
Challenge 3: Heat Stencil	Max Threads	1500 × 1500 × 300	0.094	N/A	N/A